MACHINE LEARNING · CLASIFICACIÓN

Árboles de decisión

Mapa de la clase

- 1. Problema y conceptos: clasificación, features y variable objetivo.
- 2. Cómo decide un árbol: nodos, ramas, hojas, Gini y entropía.
- 3. Notebook original: explicación celda por celda.
- 4. Evaluación ampliada: accuracy, precision, recall y F1-score.
- 5. Práctica opcional con un dataset público oficial.

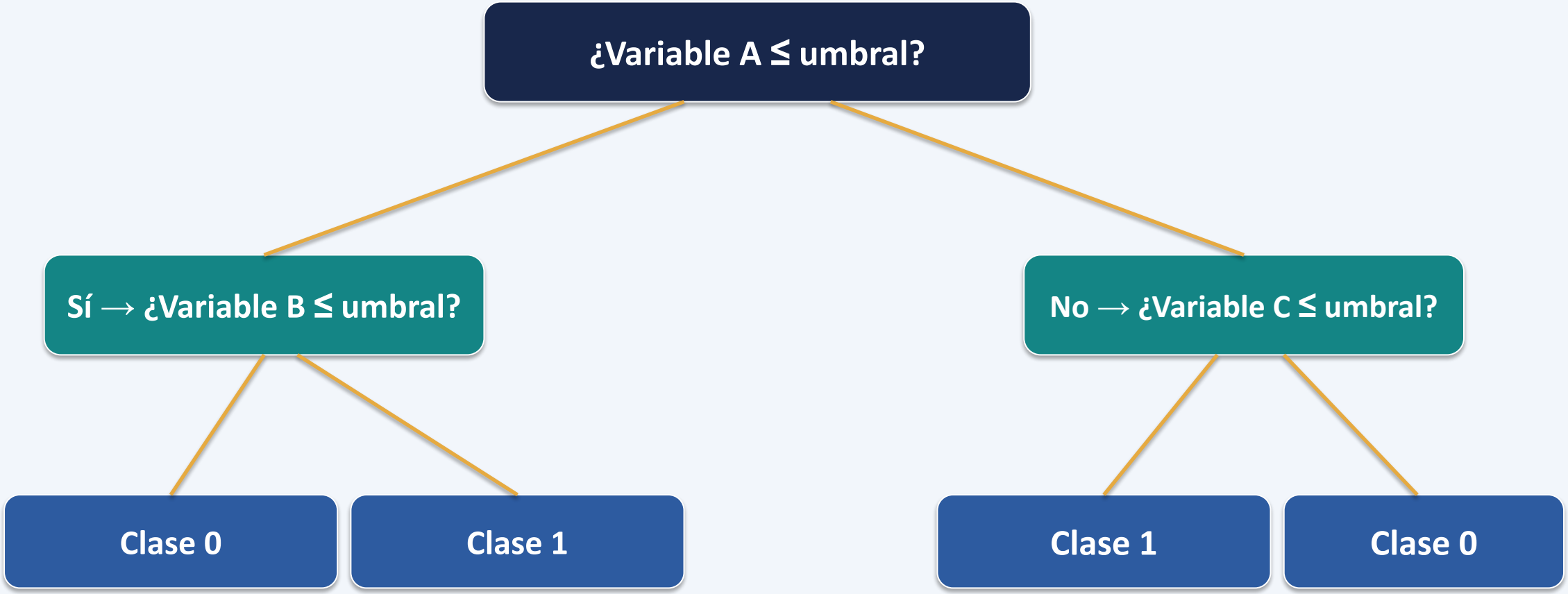
¿Qué problema resuelve la clasificación?

- Aprender a asignar una categoría a un caso nuevo usando ejemplos etiquetados.
- En el notebook, la variable objetivo es `regular`: el modelo intenta predecir su categoría.
- X (features): columnas usadas como información de entrada.
- y (target): respuesta que el modelo aprende a predecir.

¿Qué es un árbol de decisión?

- Es un modelo supervisado que organiza decisiones como preguntas sucesivas.
- Raíz: primera pregunta;
  - nodos internos: preguntas intermedias;
  - ramas: respuestas;
  - hojas: predicción final.
- Cada pregunta suele tener forma variable ≤ umbral, por ejemplo: `asistencia ≤ 75`.
- El algoritmo busca divisiones que separen mejor las clases según un criterio de impureza.
- Ventaja: es relativamente interpretable. Riesgo: un árbol demasiado profundo puede memorizar el entrenamiento.

Mecanismo de decisión: un ejemplo conceptual



Impureza: ¿qué tan mezcladas están las clases?

- Un nodo es puro si todos sus ejemplos pertenecen a una sola clase.
- Un nodo es impuro si contiene una mezcla de clases.
- El árbol compara divisiones candidatas y prefiere las que reducen más la impureza ponderada de los nodos hijos.
- Una división no tiene que producir hojas perfectamente puras para ser útil.
- Las fórmulas se calculan con proporciones de clases dentro de cada nodo.

Índice Gini

Gini = 1 − Σ p□²

- `p□` es la proporción de observaciones de la clase k dentro del nodo.
- Si un nodo contiene una sola clase, Gini = 0 (pureza máxima).
- En clasificación binaria, el máximo es 0,5 cuando las clases están 50/50.
- Ejemplo 50/50: Gini = 1 − (0,5² + 0,5²) = 0,5.
- El árbol busca una reducción de impureza al dividir el nodo.

Entropía y ganancia de información

$\text{Entropía} = -\Sigma\, p_i \log_2(p_i)$

- Entropía 0 significa que el nodo contiene una sola clase.
- En un nodo binario 50/50, la entropía es 1 bit: incertidumbre máxima para dos clases.
- Ganancia de información = entropía del padre − impureza ponderada de los hijos.
- Se elige la división que más reduce la incertidumbre, según el criterio.
- En scikit-learn se puede seleccionar `criterion='gini'` o `criterion='entropy'`.

Gini vs. entropía: ¿cuál usar?

- Ambos criterios buscan divisiones que separen las clases.
- Gini suele ser algo más sencillo computacionalmente; entropía usa logaritmos.
- En muchos conjuntos de datos producen árboles parecidos, pero no se garantiza que sean idénticos.
- El criterio se elige antes del ajuste del modelo y se puede comparar con validación.
- No hay un ganador universal: importan los datos, el objetivo y las métricas relevantes.

Profundidad, sobreajuste y control del árbol

- `max_depth=3` limita el árbol a tres niveles de decisión como máximo (según la convención de profundidad de scikit-learn).
- Un árbol poco profundo es más fácil de explicar, pero puede quedarse corto (subajuste).
- Un árbol muy profundo puede ajustarse demasiado a los ejemplos de entrenamiento (sobreajuste).
- Otros controles útiles: `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes` y poda con `ccp_alpha`.
- `random_state` ayuda a reproducir resultados cuando hay aleatoriedad.

Accuracy: qué mide y qué no

Accuracy = predicciones correctas / total de casos

- Es fácil de interpretar como proporción global de aciertos.
- Puede ser engañosa si las clases están desbalanceadas.
- Ejemplo: si 90% pertenece a una clase, predecir siempre esa clase ya logra 90% de accuracy.
- Por eso conviene mirar matriz de confusión, precision, recall y F1-score.

F1-score, precision y recall

- Precision = VP / (VP + FP): de los casos predichos positivos, ¿cuántos eran positivos?
- Recall = VP / (VP + FN): de los positivos reales, ¿cuántos detectó el modelo?
- F1 = 2 × (precision × recall) / (precision + recall): media armónica entre precision y recall.
- F1 penaliza que precision o recall sean bajos; es útil cuando importa el equilibrio entre ambos.
- En clasificación multiclase se debe especificar el promedio: `macro`, `weighted` o `micro`.

Antes de ejecutar: dos detalles del notebook original

- Celda de carga: el comentario menciona `unvime2020y2021.csv`, pero la lectura posterior busca `unvimesaludmc.csv`. Debe confirmarse cuál archivo existe.
- La variable `features_mean` toma las primeras 14 columnas originales; luego X elimina `regular`. Si esas 14 columnas incluyen la etiqueta u otra columna no deseada, los nombres pueden no coincidir con las columnas reales de X.
- La celda que usa `plt.figure(...)` necesita que `plt` esté definido como `import matplotlib.pyplot as plt`; en la celda 9 se importa `matplotlib.pyplot` sin alias, aunque antes sí se importa `plt`.
- El notebook genera dos árboles distintos: uno limitado a profundidad 3 (`clf_1`) y otro sin límite explícito (`tree`). No comparar sus dibujos como si fueran el mismo modelo.

Notebook · Celdas 2–3

```
from google.colab import files
uploaded = files.upload()

import io
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

- `files.upload()` abre un selector para subir un archivo desde la computadora a Google Colab.
- `uploaded` es un diccionario con nombre y contenido del archivo subido.
- `io.BytesIO` permite entregar bytes en memoria a pandas sin guardar primero otro archivo.

El nombre real del archivo debe coincidir con el usado en la celda de lectura.

Notebook · Celda 4: leer y mostrar el CSV

```
alumnos = pd.read_csv(

io.BytesIO(uploaded['unvimesaludmc.csv'
]),
    encoding='UTF-8',
    sep=','
)
alumnos
```

- `uploaded['unvimesaludmc.csv']` busca ese nombre exacto en los archivos cargados.
- `pd.read_csv(...)` convierte el contenido CSV en un DataFrame.
- `encoding='UTF-8'` define cómo interpretar caracteres; si hay problemas con tildes, se puede probar ISO-8859-1.
- `sep=','` indica que las columnas están separadas por comas.

Notebook · Celda 5: seleccionar nombres de variables

```
features_mean =
list(alumnos.columns[0:14])
features_mean
```

- `alumnos.columns` contiene los nombres de todas las columnas.
- `[0:14]` selecciona las columnas por posición desde la 0 hasta la 13.
- `list(...)` convierte el resultado en una lista de Python.
- Se pretende usar la lista como etiquetas del gráfico del árbol.

Más robusto: definir `features = X.columns.tolist()` después de construir X.

Notebook · Celda 6: eliminar filas con datos faltantes

```
data = alumnos.dropna()
```

- `dropna()` elimina las filas que tienen al menos un valor faltante.
- `data` es el DataFrame resultante; `alumnos` original queda intacto.
- Ventaja: simplifica el entrenamiento si el estimador no admite esos faltantes.
- Riesgo: se pueden descartar muchas filas y cambiar la composición de las clases.
- En una práctica real, contar faltantes antes y después y evaluar imputación si corresponde.

Notebook · Celda 7: separar X e y

```
X = data.drop("regular", axis=1)
y = data.regular
```

- `X` reúne las variables predictoras: todas las columnas menos `regular`.
- `axis=1` indica que se elimina una columna; `axis=0` se usa para filas.
- `y` toma la columna objetivo `regular` que el modelo aprenderá a predecir.
- Este paso evita que la etiqueta se filtre entre las variables de entrada.

Notebook · Celda 8: entrenamiento y prueba

```
from sklearn.model_selection import
train_test_split


X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.20,
    random_state=1, stratify=y
)
```

- `train_test_split` separa datos para aprender y evaluar.
- `test_size=0.20` reserva el 20% para prueba y deja el 80% para entrenamiento.
- `random_state=1` permite reproducir la partición.
- `stratify=y` intenta conservar la proporción de clases en ambos subconjuntos.
- El conjunto de prueba no debe usarse para ajustar el árbol; se reserva para evaluar.

Notebook · Celda 9: importar el clasificador y el trazado

```
from sklearn.model_selection import
train_test_split

from sklearn import tree

from sklearn.tree import
DecisionTreeClassifier

import matplotlib.pyplot
```

- `DecisionTreeClassifier` implementa el árbol de clasificación.
- `tree` da acceso a utilidades de visualización como `tree.plot_tree`.
- `train_test_split` se vuelve a importar, aunque ya se importó en la celda anterior.

Notebook · Celda 10: entrenar y visualizar el árbol limitado

```
clf_1 = DecisionTreeClassifier(
    max_depth=3, random_state=42
)
clf_1.fit(X_train, y_train)

plt.figure(figsize=(22, 12))
tree.plot_tree(
    clf_1, feature_names=features_mean,
    class_names=None, filled=True
)
plt.show()
```

- `DecisionTreeClassifier(...)` crea el modelo; `max_depth=3` limita la complejidad.
- `fit(X_train, y_train)` aprende las reglas a partir de los datos de entrenamiento.
- `figure(figsize=(22, 12))` define el tamaño del lienzo.
- `plot_tree` dibuja los nodos; `feature_names` intenta mostrar nombres de variables.
- `filled=True` colorea los nodos según la clase dominante y su pureza.
- `show()` renderiza el gráfico en pantalla.

Verificar que `len(features_mean) == X_train.shape[1]`; de lo contrario, el gráfico puede fallar o etiquetarse mal.

Notebook · Celda 11: entrenar un segundo árbol

```
from sklearn.tree import
DecisionTreeClassifier

tree = DecisionTreeClassifier(
    random_state=0
).fit(X_train, y_train)
```

- Se crea un nuevo árbol y se ajusta directamente con `.fit(...)`.
- No se especifica `max_depth`, por lo que el árbol puede crecer más hasta cumplir otros criterios de parada.
- `random_state=0` fija la semilla del modelo.
- La variable llamada `tree` reemplaza el nombre del módulo importado como `tree` en esta sesión.
- **Buena práctica**: nombrar el modelo `clf_full` y el módulo `tree` para evitar confusión.

Notebook · Celda 12: exportar el árbol a PNG

```
from sklearn.tree import export_graphviz
from pydotplus import graph_from_dot_data

dot_data = export_graphviz(
    tree, feature_names=features_mean,
    class_names=True, filled=True
)
graph = graph_from_dot_data(dot_data)
graph.write_png('tree2.png')
```

- `export_graphviz` serializa el árbol en formato DOT, un lenguaje de descripción de grafos.
- `graph_from_dot_data` convierte DOT en un grafo usando pydotplus y Graphviz.
- `write_png('tree2.png')` guarda una imagen del árbol en el entorno de ejecución.
- `pydotplus` y el sistema Graphviz deben estar instalados para que esta celda funcione.
- `class_names=True` puede no ser el uso correcto para todas las versiones; para clases legibles, pasar una lista de nombres de clase.

Si solo se quiere dibujar dentro de Colab, `plot_tree` suele ser una alternativa más simple.

Código adicional: accuracy y F1-score

```
from sklearn.metrics import (
    accuracy_score, f1_score,
    classification_report, confusion_matrix
)

y_pred = clf_1.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("F1 macro:", f1_score(
    y_test, y_pred, average="macro",
    zero_division=0
))
print(classification_report(
    y_test, y_pred, zero_division=0
))
print(confusion_matrix(y_test, y_pred))
```

- `predict(X_test)` produce las clases predichas para los casos reservados.
- `accuracy_score` calcula la proporción de predicciones correctas.
- `f1_score(..., average='macro')` calcula F1 por clase y luego promedia sin ponderar por tamaño.
- `classification_report` muestra precision, recall, F1 y soporte por clase.
- `confusion_matrix` permite ver qué clases se confunden entre sí.
- `zero_division=0` evita advertencias cuando una clase no tiene predicciones positivas; interpretar ese caso con cuidado.

Para un problema binario con una clase positiva definida, se puede usar `average='binary'` y documentar cuál es la clase positiva.

Práctica opcional: dataset oficial de UCI

- Para una actividad independiente del CSV institucional, usar el dataset Wine del UCI Machine Learning Repository.
- Contiene 178 observaciones, 13 variables químicas y una variable objetivo de tres clases de vino.
- Página oficial: https://archive.ics.uci.edu/dataset/109/wine
- Instalación: `pip install ucimlrepo`.
- Esta opción no reemplaza el dataset del notebook: sirve para practica con el mismo flujo con datos públicos y documentados.

Práctica opcional: descargar Wine desde UCI

```
# Ejecutar en Colab o en un entorno Python
!pip install ucimlrepo

from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score

wine = fetch_ucirepo(id=109)
X = wine.data.features
y = wine.data.targets.iloc[:, 0]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20,
    random_state=1, stratify=y
)
clf = DecisionTreeClassifier(
    max_depth=3, criterion="gini",
    random_state=42
).fit(X_train, y_train)
pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print("F1 macro:", f1_score(y_test, pred, average="macro"))
```

Fuente oficial UCI: https://archive.ics.uci.edu/dataset/109/wine

¿Qué hace esta celda?

- `fetch_ucirepo(id=109)` recupera el dataset Wine de UCI.
- `X` son las 13 variables predictoras; `y` es la clase de vino.
- Se divide en entrenamiento y prueba.
- Se entrena un árbol con profundidad máxima 3 y criterio Gini.
- Se informan accuracy y F1 macro, apropiado para observar el rendimiento entre las tres clases.
- En una sesión práctica, comparar `criterion='gini'` con `criterion='entropy'` manteniendo la misma partición.

Actividad

- 1. Ejecutar el notebook en orden y verificar el nombre exacto del CSV.
- 2. Mostrar cuántas filas se pierden al aplicar `dropna()`.
- 3. Entrenar un árbol con `max_depth=2`, `3` y sin límite explícito.
- 4. Comparar accuracy, F1 macro y matriz de confusión.
- 5. Cambiar `criterion='gini'` por `criterion='entropy'` y describir qué cambia.
- 6. Explicar una ruta desde la raíz hasta una hoja usando el gráfico obtenido.

Cierre

- El árbol construye reglas secuenciales para dividir observaciones en clases.
- Gini y entropía cuantifican la impureza; el algoritmo elige divisiones que la reducen.
- El flujo completo importa tanto como el modelo: carga, limpieza, X/y, partición, entrenamiento y evaluación.
- Accuracy es útil, pero F1 ayuda a evaluar el equilibrio entre precision y recall.
- Los resultados solo son confiables si el dataset, las columnas y la evaluación están bien definidos.

Referencias y recursos

- Notebook de base: `Arbol_de_Calsificacion_plot.ipynb` (archivo aportado por el docente).
- Documentación scikit-learn — Decision Trees: https://scikit-learn.org/stable/modules/tree.html
- DecisionTreeClassifier: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html
- UCI Machine Learning Repository — Wine: https://archive.ics.uci.edu/dataset/109/wine
- Nota: el CSV `unvimesaludmc.csv` no está incluido dentro del notebook aportado; se debe obtener del repositorio/fuente original del curso.

Anexo: cómo medir la impureza

Antes de mirar el código, calculemos a mano una división sencilla.

Nodo padre: 10 alumnos

Clase A: 6

Clase B: 4

Proporciones: p(A)=0,6 y p(B)=0,4

Impureza del padre

Gini = $1 - (0{,}6^2 + 0{,}4^2)$

Gini = $1 - (0{,}36 + 0{,}16) = 0{,}48$

Entropía = $-0{,}6 \log_2(0{,}6) - 0{,}4 \log_2(0{,}4) \approx 0{,}971$

La pregunta no es solo «¿cuántos ejemplos hay?», sino «¿qué tan mezcladas están las clases dentro del nodo?».

Si todas las observaciones fueran de una sola clase, ambas medidas serían 0: el nodo sería puro.

Probemos una división candidata

El árbol compara cómo quedarían los grupos después de cada pregunta posible.

Nodo padre

10 observaciones

6 de clase A · 4 de clase B

Gini = 0,48

Hijo izquierdo

5 observaciones

4 de clase A · 1 de clase B

Gini = $1 - (0{,}8^2 + 0{,}2^2) = 0{,}32$

Hijo derecho

5 observaciones

2 de clase A · 3 de clase B

Gini = $1 - (0{,}4^2 + 0{,}6^2) = 0{,}48$

La impureza de los hijos se pondera por la cantidad de observaciones.

Gini ponderado

$(5/10 \times 0{,}32) + (5/10 \times 0{,}48) = 0{,}40$

Reducción de Gini

$0{,}48 - 0{,}40 = 0{,}08$

La división reduce la impureza en 0,08.

El mismo ejemplo con entropía

La entropía también evalúa la mezcla de clases, pero usa logaritmos.

Entropía del padre

$H(\text{padre}) = -0{,}6 \log_2(0{,}6) - 0{,}4 \log_2(0{,}4)$
$\approx 0{,}971$ bits

Entropía de los hijos

Izquierdo (80/20): $H \approx 0{,}722$
Derecho (40/60): $H \approx 0{,}971$

Entropía ponderada tras dividir

$(5/10 \times 0{,}722) + (5/10 \times 0{,}971) \approx 0{,}847$

Ganancia de información

$0{,}971 - 0{,}847 \approx 0{,}124$ bits: esta división reduce la incertidumbre.

Cómo leer un nodo del gráfico de scikit-learn

Los campos visibles dependen de la configuración y de la versión.

Ejemplo ilustrativo de nodo

gini = 0,48

samples = 10

value = [6, 4]

class = A

- gini: mezcla de clases en ese nodo.
- samples: cantidad de casos que llegan al nodo.
- value: cantidad de casos por clase.
- class: clase mayoritaria / predicción del nodo.

¿Qué significa la primera pregunta?

variable_1 ≤ 7,5

- Si la condición se cumple, se sigue la rama izquierda.
- Si no se cumple, se sigue la rama derecha.
- Cada observación recorre el árbol hasta una hoja.
- La clase de la hoja es la predicción final.

Resumen

Elegir el criterio

DecisionTreeClassifier(criterion='gini')

Usa Gini para comparar divisiones.

También: criterion='entropy'

Entrenar el árbol

clf.fit(X_train, y_train)

Busca divisiones a partir de los ejemplos de entrenamiento.

Controlar complejidad

max_depth=3 limita la profundidad y ayuda a mantener el árbol interpretable.

Comprobar el resultado

plot_tree(...) permite observar reglas, impureza, muestras y clases de los nodos.

Gini y entropía no son métricas de rendimiento final; son criterios internos para elegir divisiones.